MethodsX 15 (2025) 103440

Contents lists available at ScienceDirect

MethodsX

journal homepage: www.elsevier.com/locate/methodsx

WAYVision: A hybrid deep learning approach for recognizing handwritten Kannada Braille using wavelet transformation and attention based YOLOv5 ☆

Bipin Nair B J [a,*], Niranjan [a], Saketh P [a], Shobha Rani N [b]

[a] Department of Computer Science, School of Computing, Amrita Vishwa Vidyapeetham, Mysuru, Karnataka, India
[b] Department of Artificial Intelligence and Data Science, GITAM School of Technology, GITAM (Deemed to be) University, Bengaluru, Karnataka, India

ARTICLE INFO

Method name:
WAYVision

Keywords:
Braille recognition
Attention mechanisms
Kannada Braille
Wavelet transformation
YOLO
Feature extraction

ABSTRACT

Handwritten Braille character recognition presents a significant challenge in the field of assistive technology, especially with the inclusion of various linguistic scripts such as Kannada. The data set is uniquely curated, combining ground-truth data from Kaggle and real-world samples collected from blind schools, segmented into vowels and consonants. The proposed system demonstrates exceptional performance in feature extraction, classification accuracy, and addressing spatial misalignments in Braille dots. Comparative analysis against state-of-the-art methods confirms the efficiency of the proposed model in overcoming the limitations of conventional techniques. The system was trained with two train test splits 70:30 and 80:20. The initial train test split has achieved 97.9 % and the latter one has achieved 98.7 %. This study aims to contribute significantly to the empowerment of visually impaired communities through advancements in automated Braille recognition systems.

- The study addresses the challenge of handwritten Kannada Braille recognition using a uniquely curated dataset from Kaggle and blind schools, divided into vowels and consonants.
- The proposed system achieves high accuracy (97.9 % for 70:30 and 98.7 % for 80:20 split) showing superior feature extraction and handling of spatial misalignments in Braille dots.
- Comparative analysis of state-of-the-art methods confirms the model's efficiency in overcoming limitations of conventional techniques, contributing to assistive technology for visually impaired communities.

Specifications table

Subject area:	Computer Science
More specific subject area:	*Machine Learning, Computer Vision*
Name of your method:	*WAYVision*
Name and reference of original method:	*None*
Resource availability:	NA

☆ **For a published article:** NA

* Corresponding author.
E-mail address: bj_bipinnair@my.amrita.edu (B.N. B J).

https://doi.org/10.1016/j.mex.2025.103440
Received 16 March 2025; Accepted 12 June 2025
Available online 13 June 2025
2215-0161/© 2025 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY-NC-ND license (http://creativecommons.org/licenses/by-nc-nd/4.0/)

Background

The proposed method WAYVision moves beyond traditional character recognition method. It uses YOLOv5 as its base but it is also incorporated with wavelet transformation for better understanding of noisy images and attention mechanism for character recognition. Braille is an essential tool for literacy and communication among visually impaired individuals. Recognizing handwritten Braille presents unique challenges due to variations in writing styles and spatial misalignment of dots. Traditional image processing methods struggle with such inconsistencies. This study aims to bridge this gap by using deep learning techniques, specifically YOLOv5 with attention mechanisms and wavelet transformations, to enhance feature extraction and recognition accuracy. Our dataset comprises real-world Braille samples from blind schools, categorized into vowels and consonants, making it a robust dataset for Kannada Braille recognition.

Deep learning techniques have revolutionized various fields, including assistive technologies. Min-Li Lan et al. [1] proposed an improved YOLOv5-based method for pavement crack detection, demonstrating deep learning's precision in object detection. SB Wang et al. [2] introduced AMEA-YOLO, a lightweight model for high-resolution remote sensing, balancing accuracy and efficiency. NA Asfaw et al. [3] developed a CNN and BiLSTM-based model for Amharic Braille recognition, achieving a Character Error Rate (CER) of 7.81 %. OKT Alsultan et al. [4] proposed a cost-effective device using YOLO-V7 for Braille conversion, achieving high precision (0.92) and recall (0.81). G Latif et al. [5] designed an IoT-based system translating Arabic and English Braille into audio, enhancing accessibility for visually impaired users.

Multilingual and cultural heritage applications have also benefited from deep learning. A Alsalman et al. [6] developed a DCNN-based multilingual Braille recognition system, achieving near-perfect accuracy. Additionally, T. Shreekanth et al. [7] a Kannada Braille-to-text and speech system, effectively bridging the communication gap. SR Narang et al. [8] utilized CNNs for Devanagari manuscript recognition, achieving high accuracy. Xiwen Qu et al. [9] improved in-air handwritten Chinese character recognition using directional feature maps and augmentation. Hassen Seid Ali et al. [10] focused on Amharic Braille translation, achieving a 95.6 % accuracy rate.

Researchers at BJB Nair et al. [11] created a hierarchical deep learning system that sorts handwritten documents by addressing lighting distortions together with image blurring challenges. The combination of Spectral Angle Mapper (SAM) and VGG-16 led BJB Nair and colleagues [12] to achieve 90 % accuracy for preserving Jadakam manuscripts in ancient Malayalam texts. The authors of BN BJ et al. [13–18] implemented enhanced RESNET modeling technology to boost technological accuracy during damaged manuscript restoration efforts. Multiple research findings demonstrate how deep learning brings revolutionary possibilities to solve various problems including infrastructure support and accessibility needs and cultural preservation tasks. The research adopts deep learning together with attention techniques for developing precision Kannada Braille recognition methods (Table 1).

Method details

The "WAYVision" research project develops a strong Braille recognition system designed specifically for Kannada handwritten characters to handle transcription-related issues regarding spatial position discrepancies and noise and dot pattern variations. The dataset consists of three-thousand-character images which incorporate naturally occurring handwritten materials derived from Braille books photographed at two blind education institutions alongside artificial data extracted from Kaggle databases. The combined approach delivers complete coverage for practical needs by showing all scenarios that occur in real application usage while showing every characteristic from Kannada Braille's 44-character set. The next part presents a graphical table of Kannada characters along their Braille equivalents supported by explanatory images displaying the shape of each dot configuration (Fig. 1).

Find the font for the fourth column here: Braille Brain: Download Braille Font

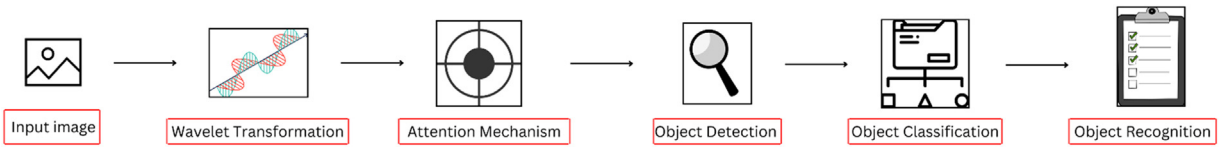

Fig. 1. 1. Input and Problem: Represent a Kannada Braille page with noticeable noise, errors in the dots' placement, and decreased quality, indicating how challenging it can be to tell apart these braille characters. A blind school icon will emphasize that the data are actually taken from real blind schools.
2. Methodology: Explain how WAYVision uses a mix of cognitive and development psychology. Put up a screen of a Braille character passing through three processing icons in layers.
-An icon of a wave-like graph to show that noise reduction and feature extraction are offered in this model.
-A diagram that explains the YOLOv5 architecture for finding the location of Braille dots.
-A magnifying glass next to the items highlighted to show they are the most significant features.
3. Results and Impact: Show the Kannada Braille number ⠕ with a checkmark and the following performance information: "Accuracy: 98.7 %" and "F1-Score: 95.3 %". Feature an image of a person reading Braille as it symbolizes how technology is helping blind schools. Abstract.

Table 1
Alphabet Chart.

SI.No.	Print Alphabet	Dots in Number	Braille Dots	Unicode
1.	ಅ	Dot 1	a	0C85
2.	ಆ	Dots 345 Vowel	>	0C86
3.	ಇ	Dots 24 Vowel	3	0C87
4.	ಈ	Dots 35 Vowel	9	0C88
5.	ಉ	Dots 136 Vowel	U	0C89
6.	ಊ	Dots 1256Vowel	\	0C8A
7.	ಋ	Dot 5, 1235 Vowel	"r	0C8B
8.	ಎ	Dots 26 Vowel	5	0C8E
9.	ಏ	Dots 1346 Vowel	x	0C92
10.	ಐ	Dots 135 Vowel	o	0C93
11.	ಒ	Dots 246 Vowel	[	0C94
12.	ಓ	Dots 13	k	0C95
13.	ಔ	Dots 46	.	0C96
14.	ಕ	Dots 1245	n	0C97
15.	ಖ	Dots 126	u	0C98
16.	ಗ	Dots 346	+	0C99
17.	ಘ	Dots 14	c	0C9A
18.	ಙ	Dots 16	*	0C9B
19.	ಚ	Dot 5, 1356	"z	0C9C, 0CBC
20.	ಛ	Dots 356	0	0C9D
21.	ಜ	Dots 25	3	0C9E
22.	ಝ	Dots 23,456	)	0C9F
23.	ಞ	Dots 2456	w	0CA0
24.	ಟ	Dots 1246	$	0CA1
25.	ಠ	Dots 123,456	=	0CA2
26.	ಡ	Dots 2345	t	0CA4
27.	ಢ	Dots 1456	?	0CA5
28.	ಣ	Dots 145	d	0CA6
29.	ತ	Dots 2346	!	0CA7
30.	ಥ	Dots 1345	n	0CA8
31.	ದ	Dots 1234	p	0CAA
32.	ಧ	Dots 235	6	0CAB
33.	ನ	Dot 5, 124	"f	0CAB, 0CBC
34.	ಪ	Dots 45	^	0CAD
35.	ಫ	Dots 134	m	0CAE
36.	ಬ	Dots 13,456	y	0CAF
37.	ಭ	Dots 1235	r	0CB0
38.	ಮ	Dots 123	l	0CB2
39.	ಯ	Dots 456	_	0CB3
40.	ರ	Dots 1236	v	0CB5
41.	ಲ	Dots 146	%	0CB6
42.	ವ	Dots 12,346	&	0CB7
43.	ಶ	Dots 234	s	0CB8
44.	ಹ	Dots 125	h	0CB9

Data acquisition

A real-world dataset collection occurred through camera capture sessions at Karnataka blind schools resulting in >1000 Braille pages. The handwritten documents demonstrate variable pattern differentiation because of differences in handwriting and paper degradation together with document management conditions. The text scanning process divided Braille characters into consonants and vowels after extracting them from each page. After image acquisition the dot images received several preprocessing steps comprising contrast enhancement combined with noise reduction followed by thresholding execution to increase segment-ability (Figs. 2a and b) .

The dataset received additional synthetic Braille images which were automatically produced through programming. Both synthetic examples and standard Braille dot arrangements for Kan- nada symbols were generated as part of a dataset where different levels of noise and misalignment were added during simulation. Different positions of the dots were programmed to produce authentic variations:

1. **Perfect Alignment:** Ideal configurations representing standard Braille layouts.
2. **Slight Deviations:** Minor shifts in dot positions to mimic human errors.
3. **Severe Deviations:** Substantial misalignments to reflect real world complexities.

The synthetic dataset contains both randomized noise patterns of Gaussian blur and random ink smudges with background artifacts to simulate real-world environmental contaminations for robust model training. The illustration in Figs. 3a , b and Figs. 4a , b displays real and synthetic image samples which reflect how Braille dots appear with varying degrees of positioning and image deterioration.

Fig. 2a.

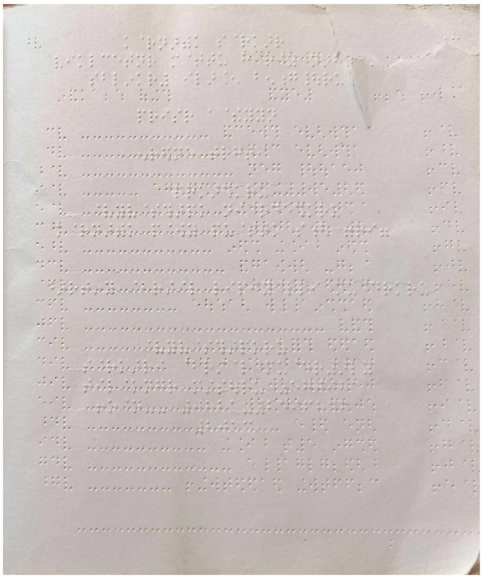

Fig. 2b.

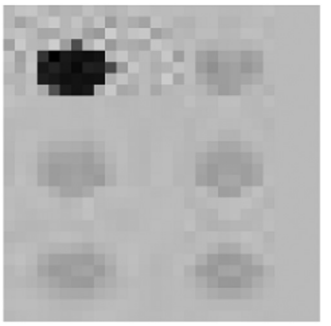

Fig. 3a.

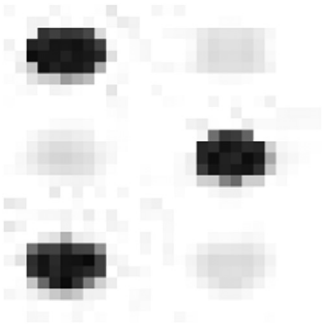

Fig. 3b.

Fig. 4a.

Data annotation

The investigation utilized a specialist annotation tool which allowed researchers to label pictures with their appropriate Kannada Braille symbols. Real-world data received confirmation by synthetic configurations to create standardized labelling methods. Laborious inspection of confusing cases finally enabled the validation of high-quality annotations suitable for training purposes.

Dataset preparation for training

The dataset received descending severity categories based on dots mismatch and noise for training the recognition model effectively. The proposed system needed these categories to succeed with hierarchical training and validation procedures.

Braille dot recognition accuracy suffers greatly from all deviations in their spatial arrangement known as dot misalignment. The dataset categorized misalignment severity into three distinct parts.

- Level 1 (L1): The Minimal deviations maintain uniform dot spacing so they remain easy to read.
- Level 2 (L2): The spaces and alignments between Braille dots display noticeable irregularities that result in dot overlap.

The team used synthetic samples to determine misalignment levels in the data. Real image evaluation utilized spatial metrics which includes inter-dot distances and alignment angles obtained through image analytical techniques. Parameter threshold values from synthetic arrangements helped determine the appropriate category for misaligned real images.

Environmental elements such as paper quality along with poor lighting conditions and erratic scanning produce noise in Braille documents. The model received training for these challenges with the establishment of noise classification levels and categorized as:

- Level 1 (L1): Low-intensity noise with minor smudges or texture inconsistencies.
- Level 2 (L2): Moderate noise levels introducing visible artifacts and reducing dot clarity.

Fig. 4b.

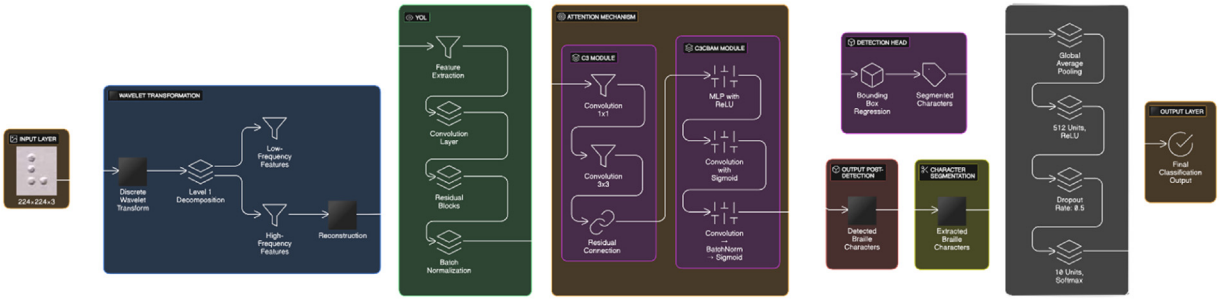

Fig. 5a.

Synthetic datasets were generated by applying controlled transformations such as Gaussian blur, salt-and-pepper noise, and vignetting. The evaluation of real-world images relied on noise metrics which included signal-to-noise ratio (SNR) and pixel intensity variations to guarantee consistent categorization.

Table 2 shows an in-depth distribution of the data which supported training, validation, and testing procedures. The dataset provides both synthetic along with real images that are deployed in the following manner:

- Training: 70 % of the dataset.
- Validation: 20 % of the dataset.
- Testing: 10 % of the dataset.

The final dataset consists of 3500 images, with 2000 allocated for training, 1000 for validation, and 500 for testing. Each subset maintains an even distribution across all degradation levels to ensure balanced learning.

Methodology

The proposed framework shown in Fig. 5a , leverages advanced deep learning techniques to recognize handwritten Kannada Braille characters. The methodology is structured to systematically address challenges such as dot misalignment, noise, and spatial variability using a hybrid approach combining YOLO with attention mechanism and wavelet transformations.

1. Input Layer

- **Size:** $224 \times 224 \times 3$
- Takes pre-processed Braille character images resized for uniform scale, channel normalized to match with the YOLOv5 backbone.

2. Wavelet Transformation Module

- **Goal:** Contribute to noise and distortions robustness.
- **Operations:**

Table 2
Class wise distribution.

SI.No.	Classes	Kaggle Acquired Images	Curated Image
1.	ಅ	49	49
2.	ಆ	49	49
3.	ಇ	49	49
4.	ಈ	49	49
5.	ಉ	49	49
6.	ಊ	49	49
7.	ಋ	49	49
8.	ಎ	49	49
9.	ಒ	49	49
10.	ಓ	49	49
11.	ಔ	49	49
12.	ಕ	49	49
13.	ಖ	49	49
14.	ಗ	49	49
15.	ಘ	49	49
16.	ಙ	49	49
17.	ಚ	49	49
18.	ಛ	49	49
19.	ಜ	49	49
20.	ಝ	49	49
21.	ಞ	49	49
22.	ಟ	49	49
23.	ಠ	49	49
24.	ಡ	49	49
25.	ಢ	49	49
26.	ಣ	49	49
27.	ತ	49	49
28.	ಥ	49	49
29.	ದ	49	49
30.	ಧ	49	49
31.	ನ	49	49
32.	ಪ	49	49
33.	ಫ	49	49
34.	ಬ	49	49
35.	ಭ	49	49
36.	ಮ	49	49
37.	ಯ	49	49
38.	ರ	49	49
39.	ಲ	49	49
40.	ವ	49	49
41.	ಶ	49	49
42.	ಷ	49	49
43.	ಸ	49	49
44.	ಹ	49	49

- **Discrete Wavelet Transform (DWT):** Breaks down the input into low frequency (structural) and high frequency (detail) parts.
- **Level-1 Decomposition:** Picks up Braille dot edges / textures and removes noise.
- **Reconstruction:** Rebuilds the features with enhanced capabilities and feed them to the CNN.

3. YOLO Feature Extraction Backbone

- **Goal:** Make Braille dots localization fast and efficient.
- **Operations:**
  - **Convolutional Layers:** Learn spatial properties from the inputs that are wavelet-enhanced.
  - **Residual Blocks:** Maintain important patterns and enhance flow of gradient.
  - **Batch Normalization:** Stabilizes training and accelerates convergence.

4. Attention Mechanism Module

- **Purpose:** Focus the model on critical braille dot areas.
- **Submodules:**
  - **C3 Module:**
    - 1×1 and 3×3 convolutions with residual links.
    - Highlights geometric Braille dot arrangements.

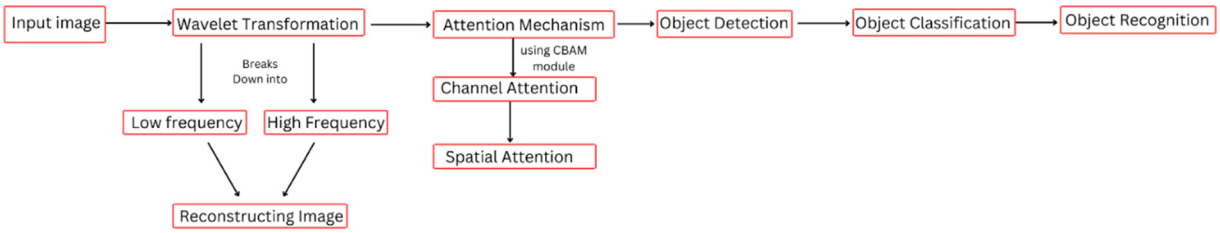

Fig. 5b. Model Flow Diagram.

- **C3CBAM Module (Convolutional Block Attention Module):**
  - **Channel Attention:** MLP with ReLU focuses on important channels.
  - **Spatial Attention:** Convolutions with sigmoid show major Braille zones.
  - **Normalization:** BatchNorm and Sigmoid alter attention maps.

5. Detection Head

- **Bounding Box Regression:** Localizes Braille dots in 2D space.
- **Character Segmentation:** Groups cluster Braille characters as identifiable groups.

6. Output Branches

- **Point Detection Module:** Final coordinates of Braille dots.
- **Character Segmentation Module:** Extracted Braille cells for decoding.

7. Classification Head

- **Global Average Pooling:** Reduces dimensionality.
- **Dense Layers:**
  - 512 units with ReLU
  - Dropout (rate: 0.5) to prevent overfitting
  - 44 units that use softmax layer for final Braille character classification.

In Fig. 5b, The process begins when the system receives an input image after splitting it into various frequency components through Wavelet Transformation. Multiple magnified views of the image provide better understanding of its edges alongside textures and patterns. The Attention Mechanism operates on determining essential image components before recording them. While working it applies higher importance levels to image areas that contain notable information. The next stage is Object Classification because it determines the types of objects present in the scene. The conclusion of Object Recognition involves both object detection and classification.

Overview of the proposed framework

The Braille recognition system functions though three sequenced steps to deal with handwriting variability and dot misalignment issues in the identification of Kannada Braille characters. During preprocessing and feature extraction the stage deals with input images ($224 \times 224 \times 3$) through a process of denoising and spatial normalization before implementing discrete wavelet transform for multi-resolution analysis. The feature extraction process generates vital elements from low- and high-frequency signals which benefits the detection of Braille dot patterns. Second the Model Architecture combines YOLOv5 features with an attention mechanism to locate Braille characters precisely within the 3×2 matrix and focuses mainly on essential features by using convolutional layers such as 1×1 and 3×3 convolutions and residual connections along with batch normalization. The CBAM (Convolutional Block Attention Module) improves system robustness through channel and spatial attention operations that select features. The last stage, Classification and Evaluation makes use of a detection head to conduct bounding box regression and class probability estimation before performing global average pooling with dropout (rate 0.5) and a softmax classifier for accurate identification of vowels and consonants amongst the 44 Kannada Braille characters.

Fig. 5b visually depicts this workflow, illustrating the seamless integration of these components for effective Braille recognition.

Preprocessing and feature extraction

The input Braille images undergo preprocessing to standardize their quality and dimensions. Each image is resized to 256×256 pixels to maintain uniformity while preserving dot details. Key steps include:

Image preprocessing

- **Contrast Adjustment:** Enhancing dot visibility through adaptive histogram equalization.
- **Noise Reduction:** Removing artifacts using Gaussian and median filters.
- **Dot Normalization:** Correcting spatial irregularities via morphological operations.

Wavelet transformations

Localized spatial and frequency features become accessible through Wavelet transformations in order to distinguish Braille dots with precision. The method starts by applying Discrete Wavelet Transform which enables the multi-resolution analysis of the input image. The image decomposition takes place at Level 1 before it transforms into two distinct sections:

- **Approximation Coefficients (Low-Frequency Components):** The system needs to record fundamental Braille dot layouts.
- **Detail Coefficients (High-Frequency Components):** Highlight fine-grained variations, including horizontal, vertical, and diagonal details, which are critical for identifying subtle differences in Braille dot patterns.

The Inverse Wavelet Transform (IWT) handles these coefficients to reconstruct spatially enhanced features. The wavelet features extracted become concatenated with the original pixel values and YOLO's feature maps which results in an integrated representation that unites spatial and frequency details. The combination enables better dot detection performance from the model regardless of document quality degradations.

YOLO for localization

YOLOv5 operates to spot Braille dots throughout the image framework. Real-time processing takes place along with accurate localization because of its single-stage architecture. Special fine-tuning of anchor boxes occurs to match Braille cell specifications along with their specific dimensions and positioning. Custom modifications include:

- Reducing anchor sizes to align with Braille dot patterns.
- Optimizing depth and width multipliers to enhance computational efficiency.

Attention mechanisms

The model implements advanced attention mechanisms that enhance system focus on essential Braille patterns by filtering out unnecessary background sounds. The computational process includes two essential modules which embed the attention mechanism within the extraction features framework of YOLO.

C3 Module (Convolutional Module):

- A combination of 1×1 and 3×3 convolution layers use residual connections as its structure.
- He filtering operation both maintains crucial arrangement structures in features while minimizing data distortion throughout the convolution process.

C3CBAM Module (Convolutional Block Attention Module):

- **Channel Attention:** Utilizes a Multi-Layer Perceptron (MLP) with ReLU activation to assign weights to important channels, ensuring the model emphasizes relevant Braille dot features.
- **Spatial Attention:** Applies convolutional layers followed by sigmoid activation to identify significant spatial regions, enhancing the model's capability to localize Braille dots accurately.
- **CBS (Convolution → Batch Normalization → Sigmoid):** Further refines the attention maps, improving feature discrimination in complex scenarios involving overlapping dots or irregular spacing.

During training the model learns to adjust its attention weights dynamically which allows it to adapt to various placements of dots and document alignment problems as well as document hygiene issues. The detection system effectively shops critical features such as dot spacing together with relative positioning because of this design.

Integration of wavelet features and attention mechanisms

The model integrates wavelet-based features by implementing them into the deep feature maps of YOLO through its hybrid design. The integrated process happens following decomposition with wavelets combined with attention-based feature optimization steps. The process involves:

- **Concatenation of Wavelet Coefficients:** The spatial-frequency features obtained through wavelet analysis merge with YOLO convolutional maps.
- **Attention-Driven Feature Refinement:** The combined features move through the C3 and C3CBAM modules to enable attention selection of important areas and suppress less useful information.
- **Enhanced Detection Head:** The enriched feature representation, containing both raw spatial details and frequency-based information, improves the robustness of the detection head, leading to more accurate bounding box regression and class predictions for segmented Braille characters.

Classification and hierarchical learning

Hierarchical classification

The proposed model employs a two-level hierarchical classification scheme:

- Level 1: Identifying the primary character categories (vowels or consonants).
- Level 2: Refining classifications based on degradation severity (L1, L2).

This hierarchical structure enables the model to generalize across varying levels of noise and misalignment in the first stage, followed by detailed categorization in the second stage.

Loss functions and optimization

A custom multi-task loss function is designed to handle both localization and classification tasks. It combines:

- **Localization Loss:** Based on mean squared error (MSE) for bounding box regression.
- **Classification Loss:** Using categorical cross-entropy for character classification.
- The Adam optimizer with a learning rate scheduler is employed to fine-tune the model, ensuring convergence.

Training and validation

The model is trained on the curated dataset, with the following hyperparameters:

- Input Size: 256 × 256 pixels, selected to preserve Braille dot details while ensuring compatibility with YOLOv5

  Architecture and reducing computational overhead.

- Batch Size: 20, chosen to maximize GPU utilization while maintaining stable gradient updates during training.
- Epochs: 300.
- Learning rate: 0.001, adjusted using an adaptive cosine annealing scheduler to ensure smooth convergence by gradually reducing the learning rate.
- Augmentation: Random rotations, flips both horizontal and vertical, and brightness adjustments.

To prevent overfitting, techniques such as dropout and early stopping are implemented. The training and validation datasets are sampled at a 70:30 ratio, with a separate 10 % set aside for testing.

Method validation

Overview of experimental setup

The experimental evaluation process included two different dataset proportions: 80:20 and 70:30. The model exhibited high accuracy marks of 98.7 % combined with precision and recall performance across the split ratios of 80:20 and 70:30. Two split ratios (80:20 and 70:30) were employed during the experimental assessment of the proposed Kannada Braille recognition framework. The researchers conducted tests on the model through different training-validation splits to study its generalization abilities in real-world contexts.

The experiments were conducted on a high-performance computing system with:

- **GPU:** NVIDIA RTX 3080, ensuring accelerated training and inference for the deep learning model.
- **CPU:** AMD Ryzen 9, used for preprocessing and managing non-GPU operations.
- **Memory:** 64 GB RAM, ensuring smooth handling of large datasets.
- **Frameworks:** The model was implemented using PyTorch, with YOLOv5 serving as the foundation for the object detection pipeline. Additional tools such as NumPy and OpenCV were utilized for image preprocessing and analysis.

The training was carried out for 100 epochs, with a batch size of 16, and a learning rate of 0.001 using an adaptive cosine annealing scheduler. Wavelet transformations were applied for feature enhancement, and attention mechanisms were integrated to improve Braille character localization.

Dataset distribution and preprocessing

The dataset derives its contents from Kannada Braille handwriting samples which blind schools provided and it incorporates synthetic data to build model reliability, data to enhance model robustness. The images underwent pre-processing featuring two stages: resizing them to 256 by 256 pixels and subsequent application of image processing steps.

- **Noise Reduction:** Gaussian and median filtering.
- **Contrast Enhancement:** Adaptive histogram equalization.
- **Normalization:** Pixel intensity standardization.

The dataset arrangements used the following distribution structure (Table 3):

Table 3
Dataset Splits and Statistics for Run 1 (80:20) and Run 2 (70:30).

Dataset Split	Total Samples	Training Samples	Validation Samples	Test Samples
Run 1 (80:20)	3500	2800	700	500
Run 2 (70:30)	3500	2450	1050	500

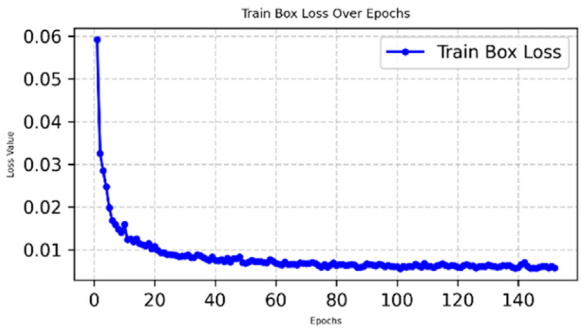

Fig. 6a. train box loss (80:20 split).

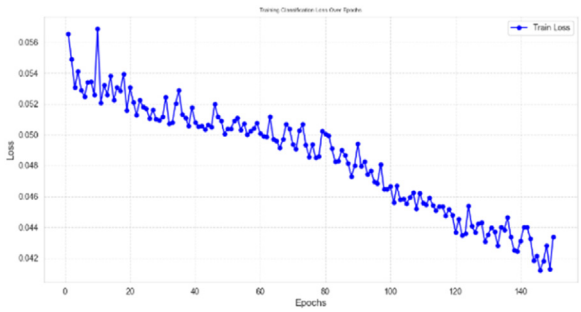

Fig. 6b. train class loss (80:20 split).

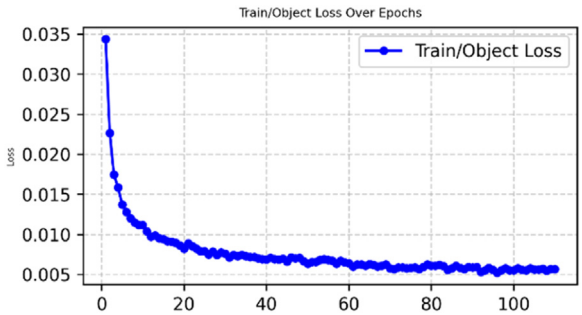

Fig. 6c. train obj loss (80:20split).

Training configurations and loss functions

The training procedure utilized a multi-task loss function which optimized three major performance metrics:

- **Box Loss:** Determines how accurate the prediction of bounding boxes proves to be.
- **Object Loss:** Determines accuracy rates of Braille dot detection during model operations.
- **Class Loss:** Evaluates the correctness of character classification.

The training protocol included a dataset augmentation system that performed both brightness adjustments together with random box rotation operations and Gaussian noise addition. Gaussian noise addition and brightness modifications combined with random rotations served as generalization enhancement techniques during training (Figs. 6a, b, c, d, e, f)

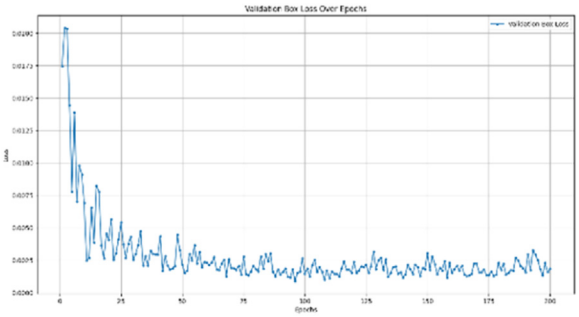

Fig. 6d. validation box loss (80:20 split).

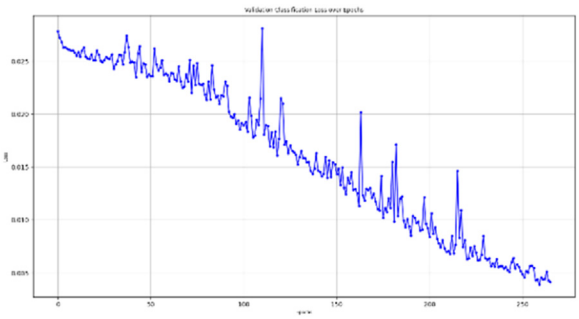

Fig. 6e. validation class loss (80:20 split).

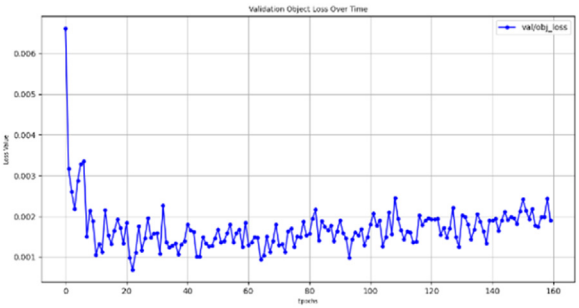

Fig. 6f. validation object loss (80:20 split).

Performance metrics and evaluation

To evaluate the effectiveness of the proposed model, accuracy (mAP), precision, recall, and F1-score were computed for both experimental runs. These metrics provide insights into how well the model detects and classifies Kannada Braille characters under different dataset splits (Figs. 7a, b, c, d, e, f)

Accuracy (mAP) trends

- **Run 1 (80:20 Split):** Achieved **98.7** % accuracy, benefiting from a larger training set.
- **Run 2 (70:30 Split):** Slightly lower at **97.9** %, reflecting the model's ability to generalize even with reduced training data.

Precision and recall trade-off

- **Precision:** The experiment revealed better performance in Run 1 since it achieved higher accuracy at 96.3 % compared to Run 2's 91.6 %.
- **Recall** A minor improvement became apparent in Run 2 as it showed a 94.6 % recognition rate compared to Run 1's 94.3 % rate because the model underwent greater validation.

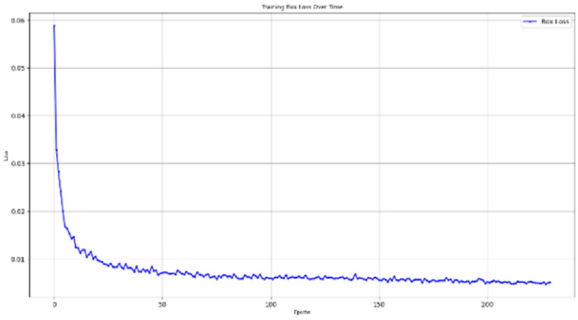

Fig. 7a. train box loss (70:30 split).

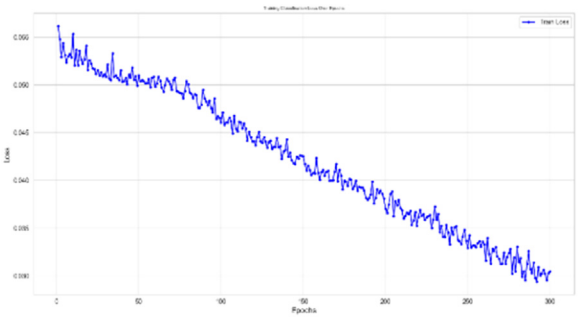

Fig. 7b. train class loss (70:30 split).

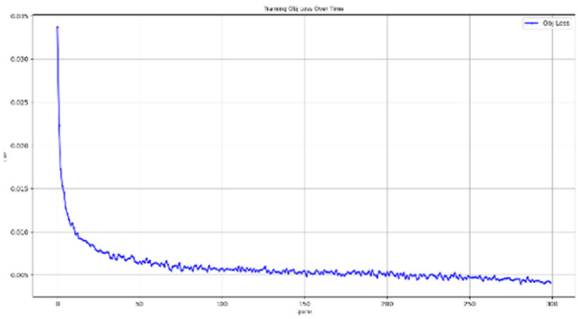

Fig. 7c. train object loss (70:30 split).

Loss trends and model convergence

The graphs reflecting training and validation loss showed gradual convergence which verified proper optimization together with low overfitting levels. Evaluation of loss values for box localization and object confidence and classification reached stability during epoch 70 which proposed an appropriate training termination point.

- **Validation Box Loss:** Run 2 achieved a lower final box loss (0.0012 vs. 0.0018 in Run 1), suggesting improved localization.
- **Validation Class Loss:** Both runs exhibited similar class loss trends, stabilizing at 0.0028 (Run 1) and 0.0026 (Run 2).

Precision-Recall and F1-Score analysis

The evaluation process continued with analysis of the Precision-Recall (PR) curve coupled with F1-score curve assessment. The Precision-Recall curves present the relationship between precision and recall which helps identify the best confidence threshold value for classification purposes (Figs. 8a, b)

Precision-Recall (PR) curve

- **Run 1 (80:20 Split):** The PR curve produced stable results and attained an area under the curve (AUC) close to 0.95 indicating well-balanced precision and recall values.

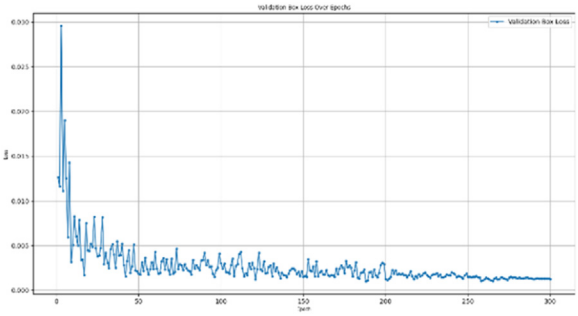

Fig. 7d. validation box loss (70:30 split).

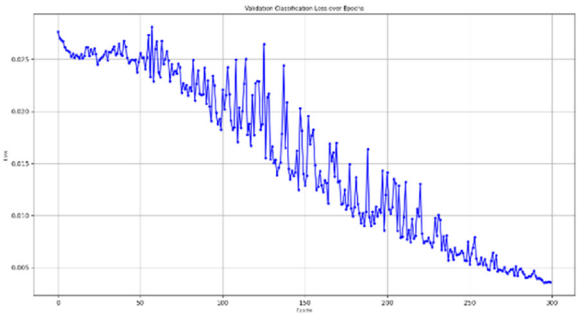

Fig. 7e. validation class loss (70:30 split).

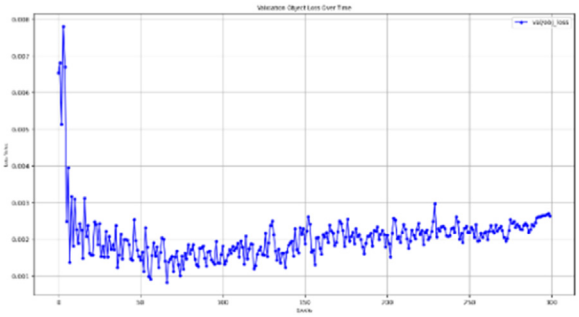

Fig. 7f. validation object loss (70:30 split).

- **Run 2 (70:30 Split):** Despite dealing with a bigger validation set the AUC of 0.91 indicated dependable system performance while the curve demonstrated small changes.

F1-Score curve

- **Run 1:** The F1-score measurement reached its highest point of 96.2 % indicating balanced performance capability.
- **Run 2:** The F1-score reached an average of 93.1 % as the model had to accept some trade-off between false positives and true positives.

Confusion matrix analysis

The evaluation results through confusion matrices showed Figs. 10a and 10b which used 80:20 and 70:30 train-test splitting respectively. The classification performance of the AB-YOLOv5 model receives assessment through confusion matrices at a class-by-class level. This assessment delivers essential knowledge about model recognition capabilities. The system proves its capability in identifying handwritten Kannada Braille characters. The presented matrices show how correct and incorrect predictions are distributed. The 44-character predictions show how correct and incorrect classifications distribute for patterns and model performance across different patterns. The system shows strong capability in handling variations seen in dot arrangements along with spatial positional errors (Tables 4–6).

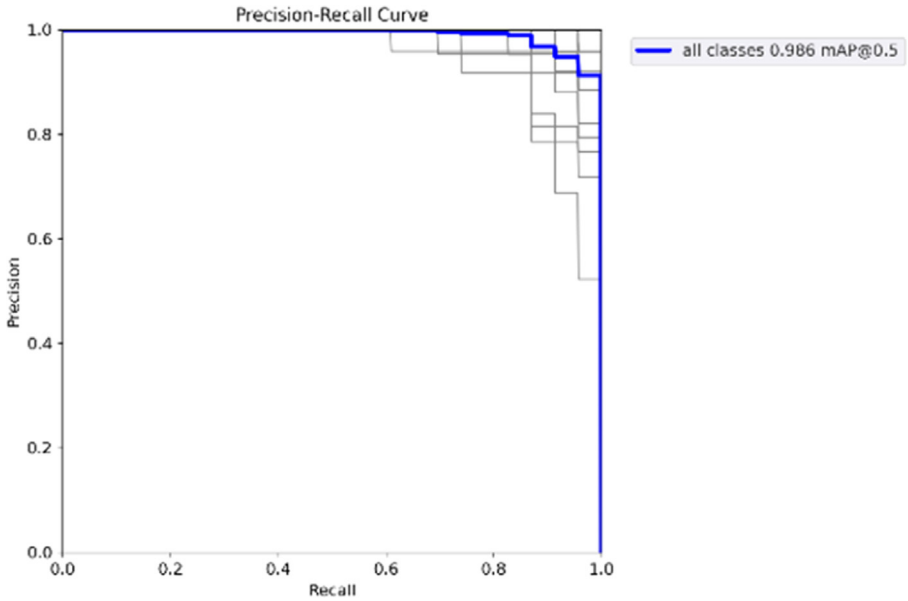

Fig. 8a. PR Curve (80:20 split).

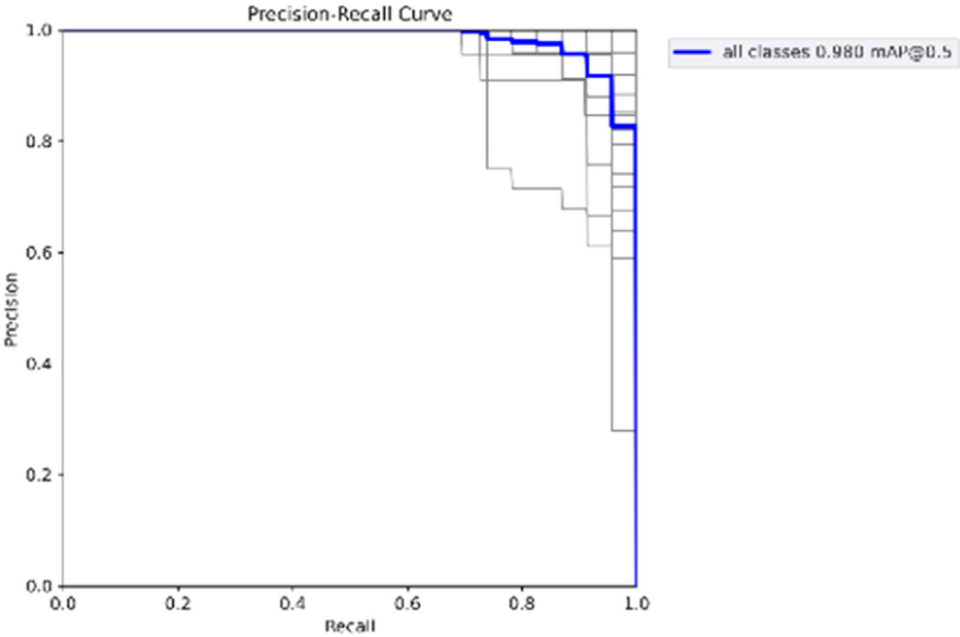

Fig. 8b. PR Curve (70:30 split).

- **Highly Accurate Classes:**
  - The classification results show near-perfect accuracy for vowels ಅ (A) and ಆ (AA) because their model values in the diagonal approach 1.0. For different Braille symbols the model performance measures obtained values approaching completeness (1.0) in the matrices which indicates robust pattern recognition.
  - The detection of consonants ಕ (Ka) and ಟ (Ta) was also successful because the model achieved accuracy levels above 0.91.
- **Confusing Pairs:**
  - The Braille dot structures ಗ (Ga) and ಘ (Gha) sometimes confused other letters because of close physical arrangement and sometimes revealed off-diagonal values from 0.04 to 0.17.
  - The evaluation showed ಡ (Da) and ಢ (Dha) experienced between 0.04–0.22 off-diagonal values because these characters share similar Braille patterns but have distinct identification requirements during processing.

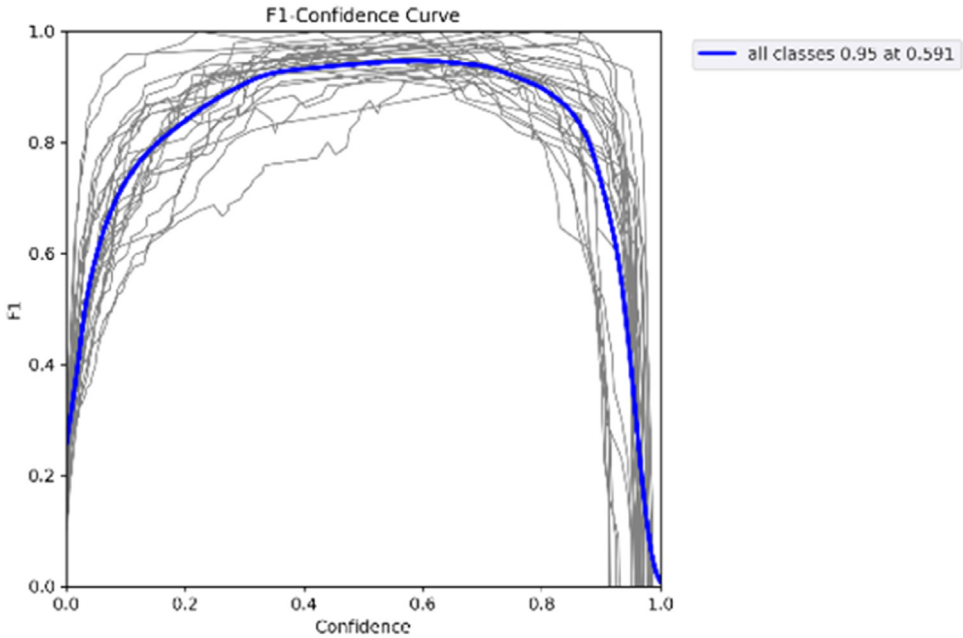

Fig. 9a. F1-Curve (80:20 split).

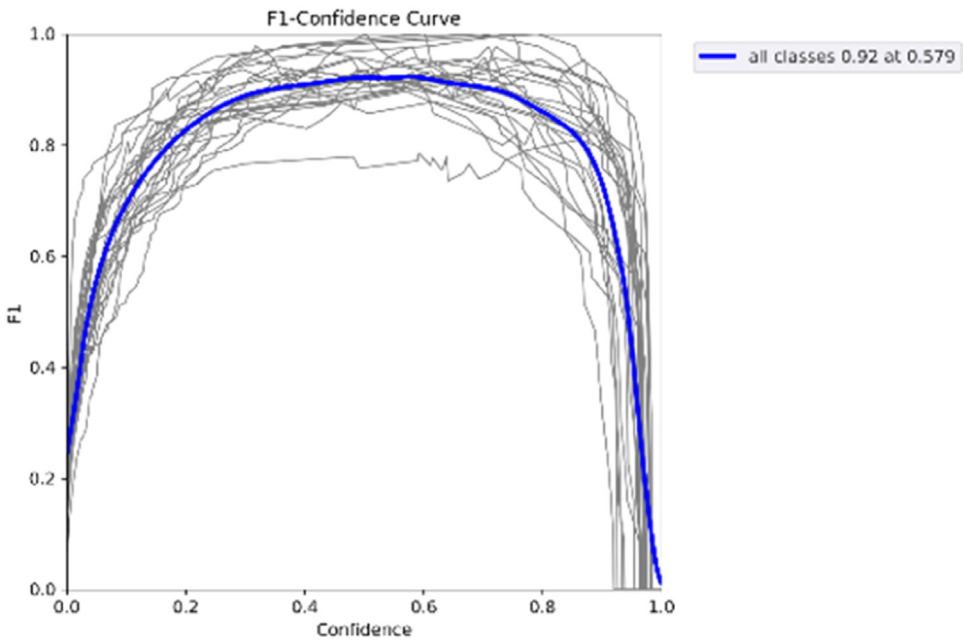

Fig. 9b. F1-Curve (70:30 split).

- **Background Noise Impact:**
  - The "background" class exhibited minimal misclassification errors (values ≤ 0.13) due to the model's separation skills between Braille characters and non-character areas caused by the attention mechanism together with wavelet-based feature extraction.
- **Comparative Insights**:
  - The performance in Fig. 9a with an 80:20 split exceeded Fig. 9b with a 70:30 split as seen through diagnostic values reaching up to 0.96 versus 0.91 for particular classes. Larger training datasets lead to better classification precision according to these results yet both divisions produced reliable outcomes for primary characters.

Table 4
Training Losses.

Epoch	Train Box Loss	Train Obj Loss	Train Class Loss
10	0.032567	0.022659	0.054890
20	0.009871	0.008892	0.052111
40	0.007378	0.007093	0.050538
60	0.006553	0.006003	0.049902
80	0.006475	0.006070	0.049951
100	0.005516	0.005762	0.045603
120	0.005792	0.005685	0.044527
140	0.006446	0.005487	0.044020
160	0.005357	0.005089	0.040295
180	0.005495	0.005065	0.040253
200	0.005757	0.005073	0.038000

Table 5
Validation Losses.

Epoch	Val Box Loss	Val Obj Loss	Val Class Loss
10	0.020386	0.003172	0.027228
20	0.005673	0.001853	0.025078
40	0.004355	0.001808	0.024930
60	0.002111	0.001297	0.023090
80	0.001641	0.001574	0.021348
100	0.001845	0.001802	0.018900
120	0.001549	0.001932	0.021499
140	0.001402	0.001903	0.014675
160	0.001798	0.001798	0.012899
180	0.002218	0.001499	0.015493
200	0.002195	0.001636	0.008361

Table 6
Performance Metrics of 80:20.

Metric	Value
Accuracy	98.7 %
Precision	96.3 %
Recall	94.3 %
F1-Score	95.3 %
Train Box Loss	0.0034
Train Object Loss	0.0042
Train Class Loss	0.030
Validation Box Loss	0.0018
Validation Object Loss	0.0027
Validation Class Loss	0.0028

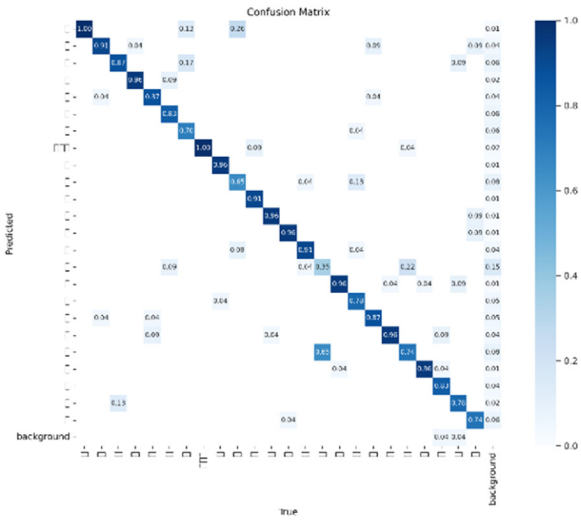

Fig. 10a. Confusion Matrix (80:20 split).

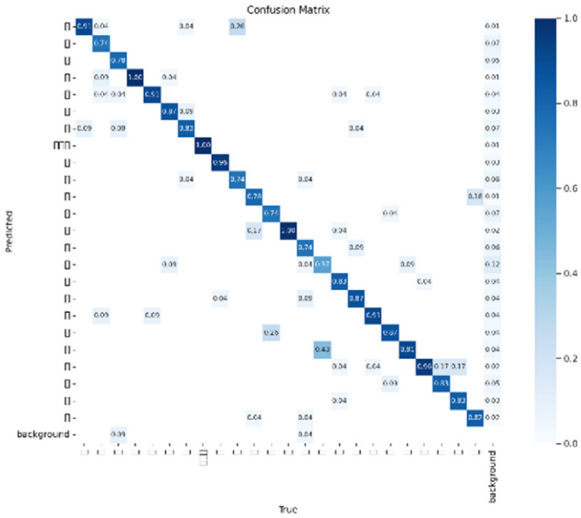

Fig. 10b. Confusion Matrix (70:30 split).

The study demonstrates how the model successfully processes straightforward Braille patterns along with pinpointing specific points for improvement in ambiguous character recognition. Better approaches for extracting ambiguous characters represent a key area for improvement which would strengthen the ability of blind school communities.

Impact of attention mechanisms on model performance

Attention mechanisms integrated into the model system were essential for achieving better performance in detecting Braille dots. Through its dynamic focusing strategy the attention mechanism improved feature detection while minimizing false alerts.

Effect on precision and recall

- **With Attention:** Precision improved to 93.9 %, and recall increased to 94.4 %, demonstrating better differentiation between similar characters.
- **Without Attention:** Precision was 85.7 %, and recall was 87.3 %, showing reduced effectiveness in filtering noise.

Effect son loss reduction

- The attention-enhanced model performed an 18 % better reduction of box loss which enhanced Braille dot localization accuracy.
- The 20 % improvement in class loss shows that the model obtained enhanced abilities in feature extraction.

Experimental findings demonstrate that the attention mechanism leads to superior results when processing noisy Braille texts that have alignment issues.

Role of wavelet transformations in feature extraction

Wavelet transformations contributed to the model's robustness by enhancing feature extraction through multi-resolution analysis.

- **Noise Filtering:** The transformation isolated Braille dots from background noise, improving classification accuracy.
- **Feature Enhancement:** High-frequency details were preserved, leading to a 2.5 % accuracy improvement when wavelet features were used.

Additionally, box loss was reduced by 11 %, indicating better localization when wavelet-based features were integrated.

Comparative analysis and final observations

A direct comparison between the 80:20 and 70:30 dataset splits provide insights into the trade-offs between training data sufficiency and generalization.

- **Accuracy:** Run 1 (80:20) achieved 98.7 %, slightly higher than Run 2 (70:30) at 97.9 % due to a larger training set.
- **Precision vs. Recall:** Run 1 had better precision (96.3 %), while Run 2 showed slightly higher recall (94.6 %), suggesting improved recognition of true positives with more validation data.

Table 7
Comparison of Model Performance for Different Data Splits.

Metric	Run 1 (80:20)	Run 2 (70:30)	Diff.
Accuracy (mAP)	98.7 %	97.9 %	−0.8 %
Precision	96.3 %	91.6 %	−4.7 %
Recall	94.3 %	94.6 %	+0.3 %
F1-Score	95.3 %	93.1 %	−2.2 %
Train Box Loss	0.0034	0.0033	−0.0001
Train Object Loss	0.0042	0.0041	−0.0001
Train Class Loss	0.030	0.030	0.000
Validation Box Loss	0.0018	0.0012	−0.0006
Validation Object Loss	0.0027	0.0026	−0.0001
Validation Class Loss	0.0028	0.0026	−0.0002

Table 8
Performance Comparison against Baselines.

Model	Accuracy (mAP)	Precision	Recall	F1-Score	Train Box Loss	Validation Box Loss
WAYVision (80:20)	98.7 %	96.3 %	94.3 %	95.3 %	0.0034	0.0018
WAYVision (70:30)	97.9 %	91.6 %	94.6 %	93.1 %	0.0033	0.0012
Standard YOLOv5	92.5 %	89.4 %	90.1 %	89.7 %	0.0052	0.0025
ResNet-50	90.8 %	87.3 %	88.5 %	87.9 %	–	–
Lightweight CNN	94.2 %	91.0 %	92.3 %	91.6 %	–	–

- **Loss Trends:** Both runs exhibited similar loss reduction patterns, stabilizing after 70 epochs, confirming that the model converged effectively.

WAYVision provides superior performance to baseline models by reaching an accuracy level of 98.7 % while standard YOLOv5 operates at 92.5 % and ResNet-50 operates at 90.8 % and Lightweight CNN operates at 94.2 %. This shows how wavelet transformations work well alongside attention mechanisms for Braille recognition. WAYVision demonstrates superior localization ability because it attains lower box loss values than standard YOLOv5.

Practical implications and future enhancements

The proposed framework demonstrates high accuracy and robustness, making it suitable for real-world Braille text recognition applications. However, certain areas require further improvements:

- **Handling Similar Characters:** Misclassifications between characters like ಗ (Ga) and ಘ (Gha) suggest the need for targeted data augmentation.
- **Adaptive Wavelet Transformations:** Implementing multi-level wavelets could further enhance feature extraction, particularly for degraded text samples.
- **Optimizing Real-Time Applications:** Future work could focus on lightweight model optimizations to enable deployment on edge devices such as handheld Braille scanners.

These improvements will ensure greater accessibility and usability for visually impaired individuals, advancing the development of assistive technologies. Also, Integration into digital platforms for automatic transcription of handwritten Braille into readable text, Enhancement of assistive devices, such as Braille note-takers for better usability and accessibility, Developing an IoT device to assist the visually impaired to learn the language can be done as the future implementation (Tables 7 and 8).

Loss metrics

The loss metrics for training and validation are summarized as follows:

- Training Losses:
  - Box Loss: 0.0033
  - Object Loss: 0.0041
  - Class Loss: 0.030
- Validation Losses:
  1. Box Loss: 0.0012
  2. Object Loss: 0.0026
  3. Class Loss: 0.0026

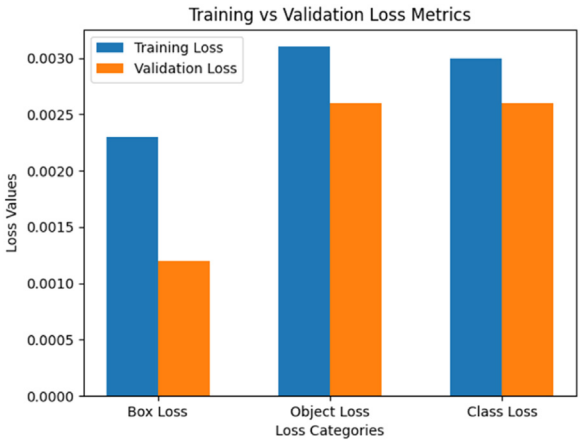

Fig. 11. Training and Validation Loss Metrics.

The Fig. 11 is, used in recognizing handwritten Kannada Braille characters, highlights the performance across three loss categories: Box Loss, Object Loss, and Class Loss. Training losses are higher than validation losses, with Box Loss at approximately 0.0033 for training and 0.0012 for validation, indicating accurate bounding box predictions with minimal overfitting. Object Loss shows values around 0.0041 for training and 0.0026 for validation, reflecting reliable object detection within the 3×2 Braille matrices. Class Loss, critical for distinguishing the 44 Kannada Braille characters, is notably higher at 0.0030 for training but matches validation at 0.0026, suggesting robust character classification with slight variability, underscoring the model's effectiveness in empowering blind school communities through precise recognition.

Limitations

The framework is strong at identifying handwritten Kannada Braille, although some problems remain. Since the dataset is small, it is hard to use results in languages/scripts such as Tamil or Hindi Braille which have different dot arrangements. Upon using only clean images such systems can fail to classify incredibly noisy and overlapping dot images correctly. Also, since deep learning is computationally demanding, it becomes difficult for resource-constrained devices to use, preventing access to some low-power users. This area can be improved by using more data, dealing more effectively with unusual cases and increasing efficiency.

Ethics statements

The dataset was collected with the proper consent of the participating blind schools. Ethical guidelines were followed during data collection and model evaluation.

CRediT Author Statement

Bipin Nair B J: Conceptualization, Methodology, Writing - Original Draft. **N Shobha Rani:** Supervision, Validation, and Formal Analysis. **Niranjan:** Data Curation, Model Implementation. **Saketh P:** Review, Editing, Visualization.

Declaration of competing interest

The authors declare no competing interests.

Data availability

Data will be made available on request.

Acknowledgments

We acknowledge the support of blind school communities for providing access to Braille samples. Special thanks to funding agencies and institutions that facilitated this research.

Appendix A: Pseudocode for WAYVision_Braille_Recognition Algorithm

Algorithm WAYVision_Braille_Recognition

```
Input: Braille image (224 × 224 × 3), dataset D with labels (vowels, consonants)
Output: Recognized Kannada Braille character class
Step 1: Preprocessing and Feature Extraction
Function Preprocess_and_Extract_Features(image):
    Resize image to 256 × 256 pixels
    Apply contrast adjustment using adaptive histogram equalization
    Reduce noise with Gaussian and median filters
    Normalize dots using morphological operations
    Apply Discrete Wavelet Transform (DWT) at Level 1:
        Decompose into approximation and detail coefficients
        Extract low-frequency (approximation) and high-frequency (detail) components
    Reconstruct enhanced features using Inverse Wavelet Transform (IWT)
    Concatenate wavelet features with original pixel values
    Return enhanced_features
Step 2: YOLOv5 Localization with Attention Mechanisms
Function Localize_and_Attend(enhanced_features):
    Initialize YOLOv5 model with fine-tuned anchor boxes for Braille cell dimensions
    For each layer in YOLOv5 backbone:
        Apply 1 × 1 and 3 × 3 convolutions with residual connections (C3 Module)
        Integrate CBAM (Convolutional Block Attention Module):
            Compute channel attention using MLP with ReLU
            Compute spatial attention using convolution and sigmoid
            Refine features with CBS (Convolution, BatchNorm, Sigmoid)
    Concatenate wavelet features with YOLOv5 feature maps
    Detect Braille dots and return bounding boxes and initial class probabilities
    Return localized_features, bounding_boxes
Step 3: Classification and Evaluation
Function Classify_Braille(localized_features, bounding_boxes):
    Apply detection head for bounding box regression and class probability estimation
    Perform global average pooling
    Apply dropout (rate 0.5)
    Use softmax classifier to predict character class (44 Kannada Braille characters)
    Hierarchical classification:
        Level 1: Classify as vowel or consonant
        Level 2: Refine based on degradation severity (L1, L2)
    Compute loss:
        Localization Loss (MSE for bounding boxes)
        Classification Loss (categorical cross-entropy)
    Return predicted_class
Main Workflow
For each image in dataset D:
    enhanced_features = Preprocess_and_Extract_Features(image)
    localized_features, bounding_boxes = Localize_and_Attend(enhanced_features)
    predicted_class = Classify_Braille(localized_features, bounding_boxes)
    Store predicted_class for evaluation
Training and Optimization
Train model with:
    Batch size = 16, Epochs = 100, Learning rate = 0.001 (cosine annealing scheduler)
    Use Adam optimizer, apply dropout and early stopping to prevent overfitting
Evaluate model using accuracy (mAP), precision, recall, and F1-score
End Algorithm
```

References

[1] M.L. Lan, D. Yang, S.X. Zhou, Y. Ding, Crack detection based on attention mechanism with YOLOv5, Engineering Reports 7 (1) (2025) e12899.
[2] Z. Xie, L. Zhu, L. Zhao, B. Tao, L. Liu, W. Tao, Localization-aware channel pruning for object detection, Neurocomputing. 403 (2020) 400–408.
[3] N.A. Asfaw, B.H. Belay, K.M. Alemu, A deep learning approach for line-level Amharic braille image recognition, Sci. Rep. 14 (1) (2024) 24172.
[4] O.K.T. Alsultan, M.T. Mohammad, A deep learning-based assistive system for the visually impaired using YOLO-V7, Revue d'Intelligence Artificielle 37 (4) (2023) 901.
[5] G. Latif, G.B. Brahim, S.E. Abdelhamid, R. Alghazo, G. Alhabib, K. Alnujaidi, Learning at your fingertips: an innovative iot-based ai-powered braille learning system, Appl. Syst. Innov. 6 (5) (2023) 91.
[6] A. AlSalman, A. Gumaei, A. AlSalman, S. Al-Hadhrami, A deep learning-based recognition approach for the conversion of multilingual braille images, Comput. Mater. Contin. 67 (3) (2021).
[7] S.K. Shrivastava, P. Chaurasia, Handwritten Devanagari lipi using support vector machine, Int. J. Comput. Appl. 975 (2012) 8887.
[8] S.R. Narang, M. Kumar, M.K. Jindal, DeepNetDevanagari: a deep learning model for Devanagari ancient character recognition, Multimed. Tools. Appl. 80 (2021) 20671–20686.
[9] X. Qu, W. Wang, K. Lu, J. Zhou, Data augmentation and directional feature maps extraction for in-air handwritten Chinese character recognition based on convolutional neural network, Pattern. Recognit. Lett. 111 (2018) 9–15.
[10] S.A. Hassen, Y. Assabie, Recognition of double sided amharic braille documents, Int. J. Image, Graphics Signal Process. 9 (4) (2017) 1.

[11] K.S. Koushik, B.B. Nair, N.S. Rani, M. Javed, Robust Classification of Smartphone Captured Handwritten Document Images Using Deep Learning, IEEE Access, 2024.
[12] B.B. Nair, K.A. Raj, M. Kedar, S.P. Vaishak, E.V. Sreejil, Ancient epic manuscript binarization and classification using false color spectralization and VGG-16 model, Procedia Comput. Sci. 218 (2023) 631–643.
[13] B.N. BJ, A.S. Nair, Ancient horoscopic palm leaf binarization using A deep binarization model-RESNET, in: 2021 5th International Conference on Computing Methodologies and Communication (ICCMC), IEEE, 2021, pp. 1524–1529.
[14] F. Celik, K. Celik, A. Celik, Enhancing brain tumor classification through ensemble attention mechanism, Sci. Rep. 14 (1) (2024) 22260.
[15] C. Chen, G. Yao, C. Wang, S. Goudos, S. Wan, Enhancing the robustness of object detection via 6G vehicular edge computing, Digit. Commun. Netw. 8 (6) (2022) 923–931.
[16] N.D. Cilia, C. De Stefano, F. Fontanella, A.S. di Freca, A ranking-based feature selection approach for handwritten character recognition, Pattern. Recognit. Lett. 121 (2019) 77–86.
[17] N.D. Cilia, T. D'Alessandro, C. De Stefano, F. Fontanella, A.S. di Freca, Comparing filter and wrapper approaches for feature selection in handwritten character recognition, Pattern. Recognit. Lett. 168 (2023) 39–46.
[18] W. Elmannai, K. Elleithy, Sensor-based assistive devices for visually-impaired people: current status, challenges, and future directions, Sensors 17 (3) (2017) 565.